INFORME PROYECTO APT

Computer Vision Walmart (CVW) — Detección y monitoreo de quiebres de stock en góndolas mediante visión por computador

Integrantes:
David Esteban Salvatierra Cayupi
David Yáñez Videla
Francisco Ignacio Fernández
Francisco Javier Merino Jofré

Carrera: Ingeniería en Informática
Institución: Duoc UC
Sede: Padre Alonso de Ovalle
Empresa asociada: Walmart Chile
Fecha: Noviembre de 2026

1 Relevancia del proyecto APT

1.1 Descripción del problema y su relevancia

En la industria del retail, la disponibilidad de productos en las góndolas es un factor importante para el funcionamiento de los supermercados y para la experiencia de compra de los clientes. Los espacios vacíos en las repisas pueden representar quiebres de stock, productos que no han sido repuestos oportunamente o situaciones que requieren la atención del personal de sala.

La detección de estos eventos suele depender de inspecciones visuales realizadas por trabajadores, lo que puede generar demoras y dificultar la identificación oportuna de los sectores que necesitan reposición.

El proyecto Computer Vision Walmart (CVW) aborda esta problemática mediante el desarrollo de un sistema de monitoreo visual que utiliza inteligencia artificial para identificar productos presentes y espacios vacíos en góndolas de supermercados.

La solución se sitúa en el contexto de las salas de venta y pasillos de supermercados, considerando las condiciones visuales propias de estos entornos: variaciones de iluminación, sombras, reflejos, diferentes ángulos de cámara, productos de distintos tamaños y cambios constantes en la disposición de las mercaderías.

1.2 Personas y áreas impactadas

El proyecto busca aportar valor a distintos actores del entorno retail:

- Personal de reposición y logística: facilita la identificación de sectores que presentan espacios vacíos y que podrían necesitar reposición.
- Administración y supervisión de tienda: permite disponer de información visual para apoyar el monitoreo de disponibilidad.
- Clientes: contribuye indirectamente a mejorar la disponibilidad de productos en las góndolas.
- Áreas de operaciones y tecnología: permite explorar la incorporación de herramientas de inteligencia artificial en procesos de supervisión y control operacional.

1.3 Aporte de valor del proyecto

El principal aporte del proyecto consiste en desarrollar una solución de software capaz de analizar imágenes de góndolas, detectar productos y espacios vacíos, calcular indicadores de disponibilidad y generar alertas cuando se cumplen determinadas condiciones.

A diferencia de una solución basada en sensores físicos instalados en cada repisa, la propuesta utiliza cámaras como fuente de información visual y un modelo de detección de objetos para interpretar las imágenes.

El sistema también incorpora una arquitectura modular que permite separar la captura de imágenes, el procesamiento mediante inteligencia artificial, la lógica de inventario y la gestión de alertas.

Durante el desarrollo se implementaron distintas fuentes de captura, incluyendo webcam, cámara USB, cámara IP mediante RTSP y una fuente sintética para pruebas.

Es importante precisar que el proyecto fue desarrollado y validado principalmente mediante conjuntos de imágenes públicos y fuentes de prueba. Por lo tanto, los resultados obtenidos representan una validación técnica del prototipo y no una implementación productiva comprobada en salas de venta de Walmart Chile.

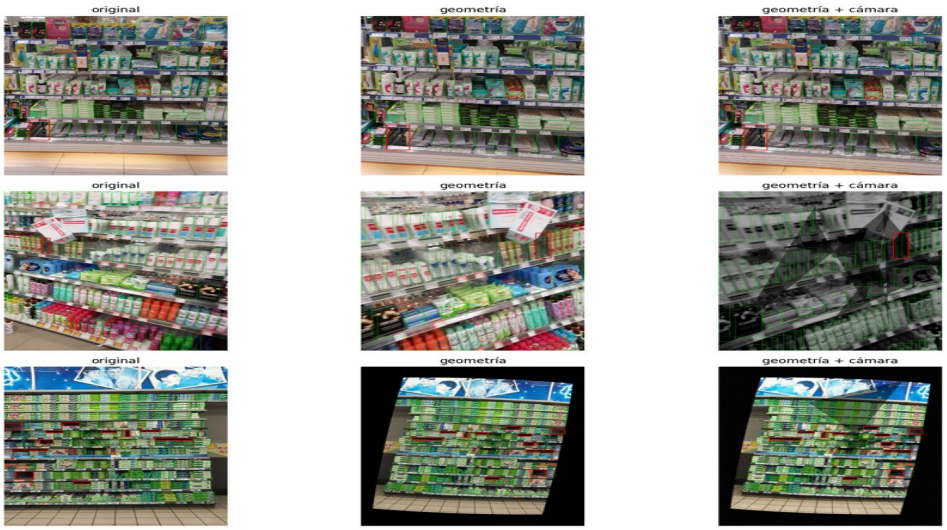
2 Objetivos

2.1 Objetivo general

Desarrollar un sistema automatizado de monitoreo e identificación de faltantes en góndolas de supermercados mediante técnicas de visión por computador e inteligencia artificial, capaz de detectar productos y espacios vacíos, calcular indicadores de disponibilidad y generar alertas ante condiciones críticas de desabastecimiento.

2.1.1 Objetivos específicos

1. Recopilar, depurar y estructurar conjuntos de imágenes de góndolas de supermercados, organizando las anotaciones en las clases product y missing para el entrenamiento y evaluación de modelos de detección.
2. Entrenar y optimizar modelos YOLO para identificar productos presentes y espacios vacíos en las repisas, evaluando diferentes configuraciones y tamaños de arquitectura.
3. Evaluar el desempeño de los modelos mediante métricas de precisión, recall, mAP50 y mAP50-95, considerando imágenes originales y versiones degradadas que simulan condiciones de cámaras de vigilancia.
4. Diseñar e implementar una arquitectura de software modular que integre captura de imágenes, inferencia del modelo, procesamiento de inventario y gestión de alertas.
5. Desarrollar una interfaz de monitoreo que permita visualizar el estado del sistema, las detecciones realizadas y los eventos generados.
6. Implementar mecanismos de evaluación y actualización de modelos que permitan incorporar nuevas muestras, revisar errores y entrenar versiones mejoradas.
7. Validar el funcionamiento del sistema mediante pruebas unitarias, de integración y de rendimiento, identificando limitaciones y oportunidades de mejora para una eventual implementación en un entorno real.



3 Metodología

3.1 Metodología utilizada

Para el desarrollo del proyecto se utilizó una metodología ágil basada en Scrum, organizada mediante etapas de trabajo iterativas y entregas incrementales.

Esta metodología permitió dividir el proyecto en actividades relacionadas con la preparación de datos, entrenamiento de modelos, desarrollo de software, integración y validación.

La elección de Scrum fue pertinente debido a la naturaleza experimental del desarrollo de modelos de inteligencia artificial. El entrenamiento de una red neuronal requiere múltiples iteraciones, ajustes de parámetros y evaluación de resultados, por lo que fue necesario adaptar las decisiones técnicas a medida que se obtenía nueva evidencia.

3.2 Fases y procedimientos desarrollados

3.2.1 Sprint 1 Adquisición y preparación de datos

Se trabajó en la recopilación y organización de imágenes de góndolas de supermercados utilizando conjuntos disponibles en Roboflow Universe.

Se utilizó el dataset *Empty Spaces in a Supermarket Hanger*, versión 4, compuesto por 349 imágenes y anotaciones de las clases product y missing.

Durante esta etapa se realizó:

- Revisión y organización de las imágenes.
- Validación de las anotaciones.
- Separación de los datos en entrenamiento, validación y prueba.
- Identificación de imágenes duplicadas o visualmente similares mediante pHash.
- Aplicación de aumentación de datos exclusivamente sobre el conjunto de entrenamiento.

La utilización de pHash permitió agrupar imágenes similares y reducir el riesgo de que fotografías prácticamente iguales quedaran distribuidas entre entrenamiento, validación y prueba, lo que podría generar resultados de evaluación artificialmente optimistas.

La partición utilizada quedó organizada de la siguiente manera:

Conjunto	Imágenes	Porcentaje aproximado
Entrenamiento	243	69,6%
Validación	53	15,2%
Prueba	53	15,2%
Total	349	100%

Además, se generaron copias degradadas de los conjuntos de validación y prueba para simular condiciones visuales similares a las de una cámara de vigilancia.

3.2.2 Sprint 2 Entrenamiento y evaluación de modelos de inteligencia artificial

Inicialmente se desarrolló un modelo basado en YOLOv8n, que permitió validar la estrategia de detección de productos y espacios vacíos.

Posteriormente, debido a las limitaciones observadas durante la evaluación, se inició el entrenamiento de una nueva versión basada en YOLOv8s, denominada YOLOv8s v2.

La elección de esta arquitectura respondió a la búsqueda de un equilibrio entre capacidad de detección y costo computacional. También se consideró que arquitecturas mayores podían presentar dificultades de generalización frente al tamaño limitado del conjunto de datos disponible.

El nuevo entrenamiento se realizó utilizando:

- Arquitectura: YOLOv8s.
- Épocas: 100.
- Resolución de entrada: 640 × 640 píxeles.
- Batch: 16.
- Hardware: GPU NVIDIA Tesla T4 mediante Google Colab.
- Congelamiento inicial de capas: 10.
- Peso del término de clasificación: 1.0.
- Máximo de detecciones: 500.

El entrenamiento tuvo una duración aproximada de 46 minutos y generó un archivo de pesos best.pt de aproximadamente 22,5 MB.

Posteriormente, se evaluó el modelo sobre conjuntos de validación y prueba, tanto originales como degradados.

3.2.3 Sprint 3 Desarrollo de la lógica de inventario

Se implementó la lógica encargada de procesar las detecciones generadas por el modelo y calcular indicadores relacionados con la disponibilidad visual de las góndolas.

La estrategia utilizada considera dos clases:

- product: productos detectados.
- missing: espacios vacíos detectados.

La proporción de espacios vacíos se calcula mediante:

missing / (product + missing)

Este cálculo corresponde a una aproximación basada en el número de detecciones de cada clase. No representa directamente el porcentaje de superficie física ocupada por los productos.

También se incorporaron mecanismos para manejar situaciones especiales, como la ausencia de detecciones, regiones de interés y condiciones que requieren validaciones adicionales.

3.2.4 Sprint 4 Desarrollo backend, frontend e integración

Se desarrolló una arquitectura modular utilizando Python y FastAPI para el backend, junto con React, TypeScript y Tailwind CSS para el frontend.

Se organizaron los componentes principales del sistema:

- Servicio de cámaras.
- Servicio de detección.
- Servicio de inventario.
- Servicio de alertas.
- Gestión de modelos.
- Gestión de muestras y datos.
- API de comunicación con la interfaz.

La solución permite utilizar diferentes fuentes de imágenes, incluyendo webcam, USB, RTSP y una fuente sintética para pruebas.

Se implementó una regla de generación de alertas que considera una tasa de espacios vacíos igual o superior al 80%, sostenida durante tres segundos, con un período de espera de 30 segundos entre alertas para evitar notificaciones repetitivas.

Los eventos pueden registrar información y capturas para facilitar su revisión. La integración con un canal definitivo de notificaciones externas permanece pendiente de confirmación.

3.2.5 Sprint 5 Pruebas, evaluación y documentación

Se realizaron pruebas de funcionamiento sobre los diferentes componentes del sistema y se evaluaron los resultados de los modelos entrenados.

Se documentaron las métricas de evaluación, los casos de mejor y peor desempeño, las limitaciones del modelo y las posibles mejoras.

También se realizaron pruebas automatizadas mediante Pytest para validar componentes de inventario, regiones de interés, ventanas temporales de alertas y gestión de modelos.

3.3 Pertinencia de la metodología

La metodología ágil permitió mantener un desarrollo progresivo y adaptar el proyecto según los resultados obtenidos.

Uno de los principales ejemplos de esta adaptación fue el cambio desde la propuesta inicial de segmentación de instancias hacia una estrategia de detección de objetos con cajas delimitadoras.

Asimismo, permitió desarrollar la arquitectura de software mientras se continuaba trabajando en el entrenamiento y evaluación de los modelos, evitando que ambas actividades dependieran completamente una de otra.

4. Desarrollo

Durante el desarrollo del proyecto se realizaron distintas actividades de investigación, preparación de datos, entrenamiento de modelos, implementación de software y evaluación técnica.

4.1 Evolución de la estrategia de detección

La propuesta inicial consideraba utilizar YOLO Segmentation para identificar la superficie ocupada por los productos y calcular el porcentaje de disponibilidad de una repisa.

Sin embargo, durante las pruebas se decidió utilizar detección de objetos mediante cajas delimitadoras, trabajando con dos clases: product y missing.

Esta modificación permitió simplificar el problema de aprendizaje y concentrar el modelo en reconocer productos presentes y espacios vacíos, en lugar de identificar individualmente cada producto comercial.

La decisión también facilitó la integración del modelo con la lógica de inventario y el resto de los servicios del sistema.

4.2 Preparación y análisis del conjunto de datos

Para el entrenamiento del nuevo modelo se utilizó la versión 4 del dataset *Empty Spaces in a Supermarket Hanger*, disponible en Roboflow Universe.

El conjunto contiene 349 imágenes y 53.278 anotaciones, distribuidas de la siguiente manera:

Clase	Cantidad de anotaciones
product	51.862
missing	1.416
Total	53.278

Se identificó un desbalance importante entre las clases, con aproximadamente 37 anotaciones de productos por cada anotación de espacio vacío.

Esta diferencia representa una dificultad para el entrenamiento, ya que el modelo dispone de muchas más muestras de productos que de espacios vacíos.

Para reducir el riesgo de fuga de información se utilizó pHash, agrupando imágenes visualmente similares antes de realizar la partición de datos.

El conjunto se dividió en 243 imágenes de entrenamiento, 53 de validación y 53 de prueba.

También se aplicaron transformaciones de aumentación sobre las imágenes de entrenamiento, incluyendo cambios geométricos, perspectiva, ruido y degradaciones visuales. Las imágenes de validación y prueba se mantuvieron separadas del proceso de aumentación de entrenamiento.

Adicionalmente, se generaron versiones degradadas de validación y prueba mediante reducción de resolución, compresión JPEG, ruido, sombras y transformaciones a escala de grises.

Estas versiones permiten estudiar cómo responde el modelo frente a condiciones visuales menos favorables. Sin embargo, constituyen simulaciones y no reemplazan la evaluación con imágenes reales de cámaras CCTV.

4.3 Entrenamiento del nuevo modelo YOLOv8s v2

Durante el proyecto se utilizó inicialmente YOLOv8n como modelo de referencia, debido a su menor tamaño y costo computacional.

Posteriormente, se entrenó una nueva versión utilizando YOLOv8s, que posee aproximadamente 11,2 millones de parámetros.

El objetivo fue estudiar si una arquitectura con mayor capacidad podía mejorar la detección, especialmente en escenarios con productos pequeños y una alta densidad de objetos.

El entrenamiento se realizó durante 100 épocas, con resolución de 640 píxeles y batch de 16, utilizando una GPU NVIDIA Tesla T4.

El modelo candidato obtuvo los siguientes resultados:

Métrica	Valid	Valid_cam	Test	Test_cam
Precisión	0,738	0,692	0,692	0,635
Recall	0,642	0,627	0,629	0,568
mAP50	0,68	0,637	0,633	0,582
mAP50-95	0,375	0,318	0,336	0,281

Al analizar los resultados por clase, se observa una diferencia importante entre la detección de productos y la detección de espacios vacíos:

Métrica por clase	Valid	Valid_cam	Test	Test_cam
mAP50 product	84,2%	77,8%	81,0%	72,7%
Recall product	78,1%	72,5%	77,9%	70,0%
mAP50 missing	51,8%	49,7%	45,6%	43,8%
Recall missing	50,2%	52,8%	47,9%	43,7%

Los resultados muestran que el modelo tiene un desempeño considerablemente mejor en la identificación de productos que en la detección de espacios vacíos.

En el conjunto de prueba original, el recall de la clase missing fue de 47,9%, mientras que en el conjunto degradado disminuyó a 43,7%.

Esto significa que todavía existe una proporción importante de espacios vacíos que el modelo no logra identificar.

Por lo tanto, aunque el nuevo modelo representa una alternativa para continuar mejorando la solución, sus resultados todavía no permiten considerarlo una versión definitiva para reemplazar al modelo integrado.

4.4 Comparación entre YOLOv8n y YOLOv8s v2

Durante el desarrollo se utilizaron dos arquitecturas principales:

Característica	YOLOv8n	YOLOv8s v2
Arquitectura	Nano	Small
Parámetros aproximados	3 millones	11,2 millones
Dataset	Roboflow v3	Roboflow v4
Cantidad de imágenes	Aproximadamente 199-200	349
Evaluación documentada	Validación	Prueba
mAP50	72,8%	63,3%
mAP50-95	38,8%	33,6%

Es importante señalar que estos resultados no representan una comparación directa entre ambos modelos, debido a que se utilizaron versiones diferentes del dataset y metodologías de partición distintas.

El modelo YOLOv8n fue evaluado sobre una partición aleatoria de una versión anterior del conjunto de datos, mientras que YOLOv8s v2 fue evaluado mediante una partición agrupada por similitud perceptual, diseñada para reducir la presencia de imágenes casi duplicadas entre entrenamiento y prueba.

Por esta razón, no es correcto concluir que YOLOv8s v2 es necesariamente inferior a YOLOv8n.

Para determinar cuál presenta un mejor desempeño real, ambos modelos deben evaluarse sobre las mismas imágenes, con idénticas condiciones de inferencia y los mismos criterios de medición.

4.5 Análisis de errores y casos de evaluación

Se analizaron las 53 imágenes del conjunto test_cam, utilizando un umbral de confianza de 0,38 y un criterio de correspondencia de IoU igual o superior a 0,5.

El análisis permitió identificar casos donde el modelo presentó buenos resultados y otros donde tuvo dificultades importantes.

Entre los casos con mejor desempeño se observaron imágenes frontales, con buena iluminación, una góndola claramente visible y espacios vacíos bien definidos.

En cambio, los casos con peor desempeño presentaron condiciones como:

- Imágenes con sombras intensas.
- Fotografías en escala de grises.
- Ángulos de observación oblicuos.
- Góndolas alejadas de la cámara.
- Productos pequeños y muy juntos.
- Fondos metálicos, rejillas y texturas que generan falsas detecciones.

Uno de los principales problemas identificados fue la dificultad para distinguir espacios vacíos reales de elementos del fondo de la góndola.

En el análisis de las diez imágenes con peor resultado se detectó solamente uno de los 32 espacios vacíos anotados, lo que evidencia que la clase missing necesita mejoras importantes.

Este análisis permitió comprender que el desempeño general del modelo no es suficiente para evaluar su utilidad operacional. Es necesario considerar especialmente la capacidad de detectar los espacios vacíos, ya que estos son fundamentales para la generación de alertas.

4.6 Evaluación exploratoria de SAM

Durante el proyecto también se exploró SAM como alternativa para obtener máscaras de segmentación a partir de las detecciones generadas por YOLO.

En las pruebas realizadas se observó que las máscaras presentaban una fuga mediana aproximada del 10% fuera de las cajas delimitadoras.

Sin embargo, la ejecución tuvo una latencia aproximada de 503 milisegundos por imagen en una GPU NVIDIA T4, considerablemente superior a los aproximadamente 9-20 milisegundos reportados para la inferencia YOLO en ese entorno.

Además, no se disponía de máscaras manuales de referencia para determinar si la segmentación generada por SAM era realmente más precisa.

Por estas razones, se decidió no incorporar SAM al flujo estándar del sistema. Su utilización queda como una línea de investigación futura, condicionada a una validación mediante anotaciones manuales de segmentación.

4.7 Arquitectura e implementación del sistema

El sistema se desarrolló utilizando una arquitectura modular, separando las responsabilidades de procesamiento y visualización.

Backend:

Implementado mediante Python y FastAPI. Contiene los servicios de captura de imágenes, detección, inventario, alertas y gestión de modelos.

Frontend:

Implementado mediante React, TypeScript y Tailwind CSS. Permite visualizar el funcionamiento del sistema y consultar la información generada durante el monitoreo.

Captura de imágenes:

Se implementaron diferentes fuentes:

- Webcam.
- Cámara USB.
- Cámara IP mediante RTSP.
- Fuente sintética para pruebas.

Procesamiento:

El flujo general del sistema corresponde a:

Fuente de imagen → Captura → Detección YOLO → Filtrado ROI → Inventario → Evaluación de alertas

Gestión de alertas:

Se implementó una condición de alerta cuando la tasa de espacios vacíos alcanza o supera el 80% durante tres segundos consecutivos.

Además, se incorporó un cooldown de 30 segundos para evitar la generación excesiva de eventos repetidos.

4.8 Pruebas y calidad del software

Se desarrollaron pruebas automatizadas mediante Pytest para validar diferentes componentes del backend.

Entre las funcionalidades evaluadas se encuentran:

- Cálculo de indicadores de inventario.
- Manejo de casos de división por cero.
- Filtrado mediante regiones de interés.
- Validación de ventanas temporales de alertas.
- Control de cooldown.
- Gestión de modelos.
- Ciclo de vida de muestras para evaluación y reentrenamiento.

El proyecto cuenta con una base de pruebas que permite verificar el comportamiento de los componentes principales y facilitar futuras modificaciones.

4.9 Principales dificultades y decisiones técnicas

Durante el desarrollo se identificaron distintas dificultades que requirieron modificar o complementar la planificación inicial.

Limitación de datos: el conjunto disponible contiene un número reducido de imágenes y un importante desbalance entre las clases product y missing. Se incorporaron técnicas de

aumentación y una partición basada en similitud perceptual para mejorar la calidad del proceso experimental.

Dificultad en la detección de espacios vacíos: el modelo presenta menor capacidad para identificar la clase missing. Se identificó la necesidad de recopilar más ejemplos y mejorar la diversidad de las anotaciones.

Variabilidad visual: las sombras, reflejos, ángulos y condiciones de iluminación afectan la detección. Para estudiar estos problemas se generaron conjuntos de imágenes degradadas, aunque será necesario validar posteriormente con imágenes reales.

Cambio de estrategia de IA: se abandonó la idea inicial de utilizar segmentación de instancias como enfoque principal y se optó por detección mediante cajas delimitadoras, debido a su menor complejidad y mayor facilidad de integración.

Actualización del modelo: se entrenó YOLOv8s v2 como alternativa al modelo YOLOv8n. Sin embargo, debido a las diferencias entre los conjuntos de evaluación y a los resultados obtenidos, se determinó que todavía es necesario realizar una comparación controlada antes de aprobar el reemplazo.

5. Evidencias

Las evidencias del proyecto permiten demostrar tanto el desarrollo de la solución de software como el proceso de entrenamiento y evaluación de los modelos.

Se consideran las siguientes evidencias:

Evidencia	Descripción	Objetivo
Dataset preparado	Imágenes y anotaciones de las clases product y missing, con particiones de entrenamiento, validación y prueba.	Demostrar el proceso de preparación de datos.
Proceso de entrenamiento YOLOv8n	Resultados y métricas del modelo de referencia.	Evidenciar el desarrollo de la primera versión del detector.
Entrenamiento YOLOv8s v2	Notebook, configuración, resultados y pesos del nuevo modelo.	Demostrar el proceso de actualización y evaluación del modelo.
Métricas de evaluación	Tablas de precisión, recall, mAP50 y mAP50-95.	Justificar técnicamente el desempeño de los modelos.
Análisis de detecciones	Imágenes con cajas delimitadoras, predicciones correctas y errores identificados.	Mostrar el comportamiento del modelo en diferentes condiciones.
Arquitectura del backend	Estructura de servicios y rutas de la API.	Evidenciar la modularización del sistema.
Interfaz del sistema	Capturas del frontend y visualización de detecciones.	Mostrar la integración y presentación de la solución.
Pruebas automatizadas	Resultados de ejecución de Pytest.	Evidenciar la validación de los componentes del software.
Demostración funcional	Ejecución del sistema utilizando una fuente de imágenes o video.	Demostrar el flujo de captura, detección, inventario y alertas.

Evidencias visuales recomendadas para adjuntar al formulario:

1. Captura de una imagen procesada por YOLOv8n con las detecciones de product y missing.
2. Captura de una imagen procesada por YOLOv8s v2.
3. Gráficos de entrenamiento y validación del nuevo modelo.
4. Tabla o matriz de resultados del conjunto de prueba.
5. Ejemplo de una detección correcta y un ejemplo de una detección fallida.
6. Captura de la interfaz principal del CVW Inventory Monitor.

7. Captura de la documentación de la API.
8. Resultado de ejecución de las pruebas automatizadas.

6. Intereses y proyecciones profesionales

6.1 Reflexión sobre el aporte del proyecto

El desarrollo del proyecto Computer Vision Walmart permitió aplicar los conocimientos adquiridos durante la carrera en una solución que combina inteligencia artificial, procesamiento de imágenes y desarrollo de software.

Durante el proceso se pudo comprender que entrenar un modelo de inteligencia artificial es solamente una parte de la construcción de una solución funcional. También es necesario preparar correctamente los datos, evaluar las predicciones, analizar los errores, desarrollar una arquitectura que permita integrar el modelo y construir mecanismos para utilizar sus resultados dentro de un proceso operacional.

Uno de los aprendizajes más importantes fue comprender la diferencia entre obtener buenas métricas durante el entrenamiento y conseguir un modelo que se comporte correctamente frente a imágenes nuevas y condiciones visuales diferentes.

La evaluación del nuevo modelo YOLOv8s v2 permitió profundizar en aspectos como el desbalance de clases, la fuga de información entre conjuntos de datos, la importancia de las particiones de prueba y las limitaciones de las métricas generales cuando una clase tiene menor representación.

También permitió experimentar con técnicas de aumentación de datos, evaluación de imágenes degradadas y herramientas complementarias como SAM.

Por otra parte, la implementación del backend y frontend permitió fortalecer conocimientos de arquitectura de software, integración de servicios, desarrollo de APIs y pruebas automatizadas.

6.2 Evolución de los intereses profesionales

El proyecto permitió confirmar y ampliar los intereses profesionales del equipo en áreas relacionadas con Inteligencia Artificial, Machine Learning, Deep Learning, desarrollo backend y arquitectura de software.

Además, se identificó el interés por profundizar en el desarrollo de soluciones donde los modelos de inteligencia artificial no solamente se entrenen, sino que también puedan integrarse, evaluarse, mantenerse y actualizarse dentro de sistemas reales.

6.3 Proyecciones laborales

A partir de la experiencia obtenida, se identifican distintas áreas de desarrollo profesional:

- Machine Learning Engineering: entrenamiento, evaluación y optimización de modelos de aprendizaje automático.
- MLOps: gestión del ciclo de vida de modelos, automatización de entrenamientos, despliegue y monitoreo.
- Computer Vision: desarrollo de soluciones para detección de objetos, análisis de imágenes y procesamiento de video.
- Edge AI: optimización de modelos para su ejecución en dispositivos y equipos locales.
- Backend y arquitectura de software: desarrollo de APIs, servicios y sistemas modulares que integren componentes de inteligencia artificial.

- Analítica operacional: utilización de información generada por modelos para apoyar procesos de negocio.

A corto y mediano plazo, el equipo busca proyectarse en roles relacionados con ingeniería de software, desarrollo backend, ingeniería de Machine Learning y áreas de inteligencia artificial aplicada.

La experiencia obtenida durante el proyecto constituye una base para continuar profundizando en estas tecnologías y participar en el desarrollo de soluciones que permitan automatizar procesos y resolver problemas concretos de la industria.

7. Conclusiones

El proyecto Computer Vision Walmart permitió desarrollar una solución de monitoreo visual orientada a la detección de productos y espacios vacíos en góndolas de supermercados, utilizando técnicas de visión por computador e inteligencia artificial.

Durante el desarrollo se pasó de una propuesta inicial basada en segmentación de instancias a una estrategia de detección de objetos mediante dos clases: product y missing. Esta decisión permitió simplificar el problema y facilitar la integración del modelo con la lógica de inventario y los servicios del sistema.

Se desarrolló una primera versión utilizando YOLOv8n y posteriormente se entrenó un nuevo modelo experimental basado en YOLOv8s v2. Esta última versión utilizó un conjunto de datos de 349 imágenes, una partición basada en similitud perceptual y un proceso de evaluación sobre imágenes originales y degradadas.

Los resultados obtenidos permitieron identificar avances y limitaciones. En particular, el nuevo modelo presenta un mejor desempeño en la detección de productos que en la identificación de espacios vacíos. El recall de la clase missing en el conjunto de prueba fue de 47,9%, lo que demuestra que todavía es necesario mejorar la capacidad del sistema para reconocer faltantes.

También se desarrolló una arquitectura modular que integra captura de imágenes, inferencia, procesamiento de inventario, gestión de alertas y visualización mediante una interfaz web. Se incorporaron pruebas automatizadas para validar componentes importantes del backend.

Uno de los principales aprendizajes fue comprender que la calidad de un modelo no depende solamente de su arquitectura o del número de épocas de entrenamiento, sino también de la calidad y diversidad de los datos, la metodología de evaluación y las condiciones en las que será utilizado.

Finalmente, el proyecto permitió cumplir una parte importante de los objetivos de desarrollo e integración planteados, dejando una base funcional y documentada para continuar mejorando la solución.

El reemplazo definitivo de YOLOv8n por YOLOv8s v2 queda condicionado a una evaluación comparativa bajo las mismas condiciones y al cumplimiento de criterios de aceptación. Asimismo, será necesario incorporar imágenes reales de cámaras de supermercados para validar su comportamiento en un entorno operacional.

En conclusión, el proyecto representa una experiencia de aplicación de inteligencia artificial y desarrollo de software a una problemática del sector retail, permitiendo al equipo fortalecer competencias técnicas y adquirir experiencia en el desarrollo de soluciones integradas de visión por computador.